

Design and Analysis of Algorithms

Lecture 4: Asymptotic Analysis (2)



Excerpts from the previous lectures

- We primarily analyze Time complexity (runtime) over Space complexity.
- We use Theoretical (Asymptotic) Analysis because the Naïve approach is unreliable (dependent on hardware, implementation, and specific input data).
- Time Complexity is calculated by counting the number of primitive operations (such as addition, indexing, comparison) as a function of the input size.



Excerpts from the previous lectures

- The analysis of the complexity of a recurrence relation involves finding the asymptotic upper bound on the running time of a recursive algorithm. This is usually done by finding a closed-form expression for the number of operations performed by the algorithm as a function of the input size and then determining the order of growth of the expression as the input size becomes large.
- Solving recurrences plays a crucial role in the analysis, design, and optimization of algorithms, and is an important topic in computer science. There are mainly three ways of solving recurrences:
 - Substitution Method
 - Master Theorem Method
 - Tree Method



Tree Method

- **Important Rules for Recursion Tree Method**

1. Increasing Geometric Series ($r > 1$)

General Form:

$$S = a_1 + a_1r + a_1r^2 + \dots + a_1r^{h-1}$$

Sum Formula:

$$\sum = a_1 \left(\frac{r^h - 1}{r - 1} \right)$$

Alternative Form:

$$\sum = a_1 \left(\frac{1 - r^h}{1 - r} \right)$$

Asymptotic Behavior:

- Dominant term: a_1r^{h-1} (last term)
- Complexity: $O(r^h)$



Tree Method

- **Important Rules for Recursion Tree Method**

2. Decreasing Geometric Series ($r < 1$)

General Form:

$$S = a_1 + a_1r + a_1r^2 + \dots + a_1r^{h-1}$$

Sum Formula:

$$\sum = a_1 \left(\frac{1 - r^h}{1 - r} \right)$$

For infinite decreasing series ($h \rightarrow \infty$):

$$\sum = \frac{a_1}{1 - r}$$

Asymptotic Behavior:

- Dominant term: a_1 (first term)
- Complexity: $O(a_1)$



Tree Method

- **Important Rules for Recursion Tree Method**

3. Constant Series ($r = 1$)

General Form:

$$S = a_1 + a_1 + a_1 + \cdots \quad (\text{h terms})$$

Sum Formula:

$$\sum = a_1 \cdot h$$

Asymptotic Behavior:

- Complexity: $O(h)$



Tree Method

- Important Rules for Recursion Tree Method

4. Exponential-Logarithm Inverse Rule

$$a^{\log_b c} = c^{\log_b a} \quad \text{Example: } 2^{\log_2 n} = n^{\log_2 2} = n^1 = n$$

5. Arithmetic Series

$$\sum_{i=1}^n i = \frac{n(n+1)}{2} = O(n^2)$$

6. Harmonic Series

$$\sum_{i=1}^n \frac{1}{i} = O(\log n)$$



Tree Method

- Instead of starting from the recursive calls, we begin with the non-recursive work $f(n)$ and build the tree downward.
- **Steps to Build the Recursion Tree**
 1. Start with the root
 - The root represents the **original problem** of size n .
 - At the root, we place the **non-recursive part of the cost**, usually $f(n)$.
(This is the direct work done at that level before making recursive calls).



Tree Method

2. Create the children

- For each recursive call in the recurrence, draw a **child node**.
- The child node corresponds to a **smaller subproblem** (like $T(n/2)$, $T(n - 1)$, etc.).
- At each child node, again separate:
 - **Non-recursive cost** (stays at that node)
 - **Recursive calls** (become its children)

3. Repeat the expansion

- Keep expanding each node into its children until the subproblem reaches a **base case** (like size 1, where cost is constant).
- At that point, no more children are drawn.



Tree Method

4. Compute cost per level

- Each level of the tree has multiple nodes.
- The **total cost at a level** = sum of the non-recursive costs of all nodes at that level.

5. Compute the total cost

- Add up the costs across all levels:

$$T(n) = \text{Cost at level 0} + \text{Cost at level 1} + \cdots + \text{Cost at last level}$$



Tree Method

- Solve this recurrence relation using tree method: $T(n) = T\left(\frac{n}{4}\right) + T\left(\frac{n}{2}\right) + n^2$

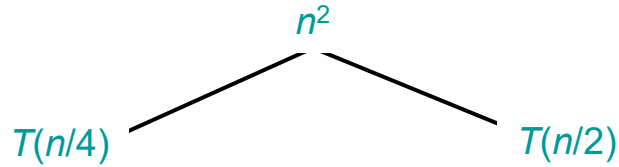
$T(n)$

Height of the tree=
 $\max(\log_4 n, \log_2 n)$



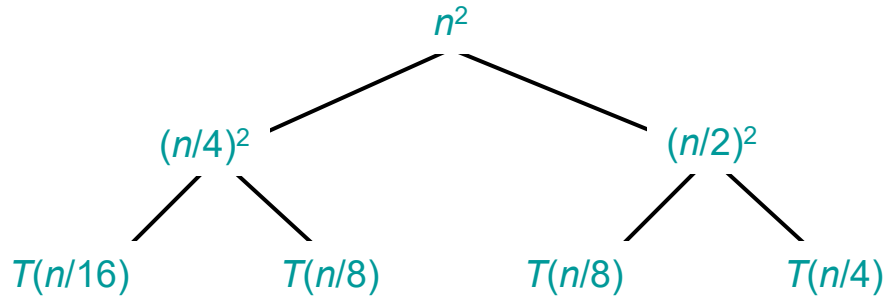
Tree Method

- Solve this recurrence relation using tree method: $T(n) = T\left(\frac{n}{4}\right) + T\left(\frac{n}{2}\right) + n^2$



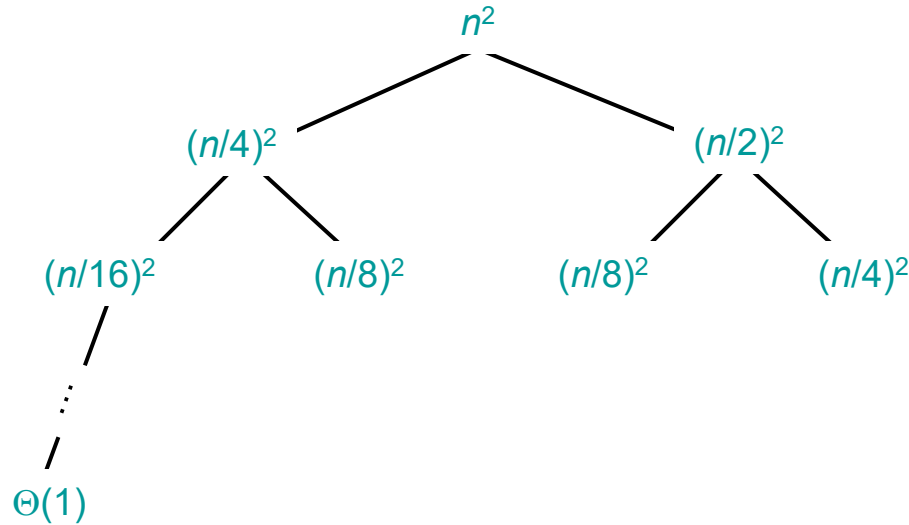
Tree Method

- Solve this recurrence relation using tree method: $T(n) = T\left(\frac{n}{4}\right) + T\left(\frac{n}{2}\right) + n^2$



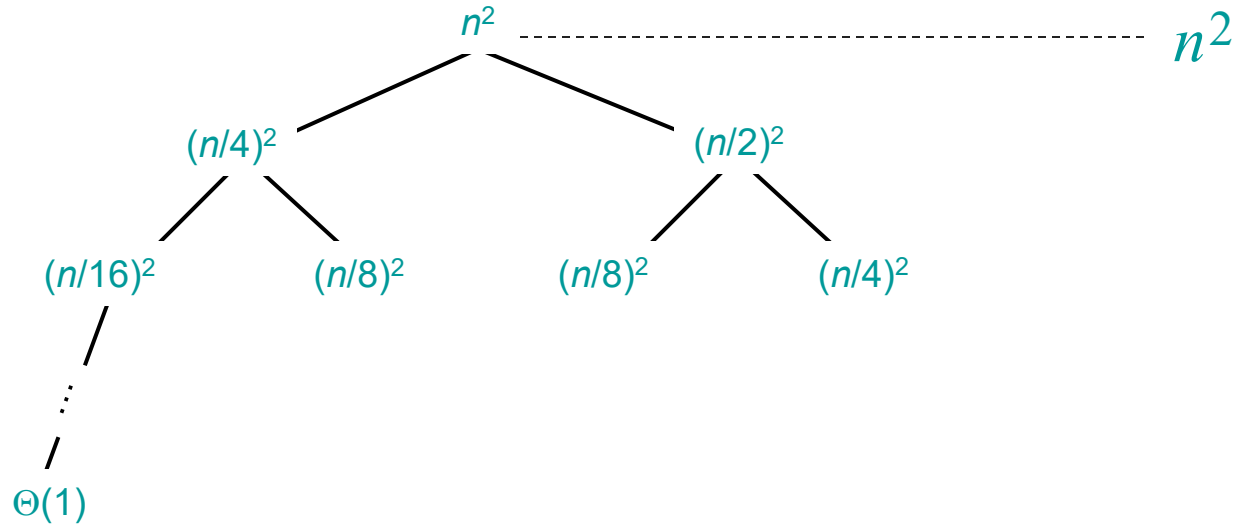
Tree Method

- Solve this recurrence relation using tree method: $T(n) = T\left(\frac{n}{4}\right) + T\left(\frac{n}{2}\right) + n^2$



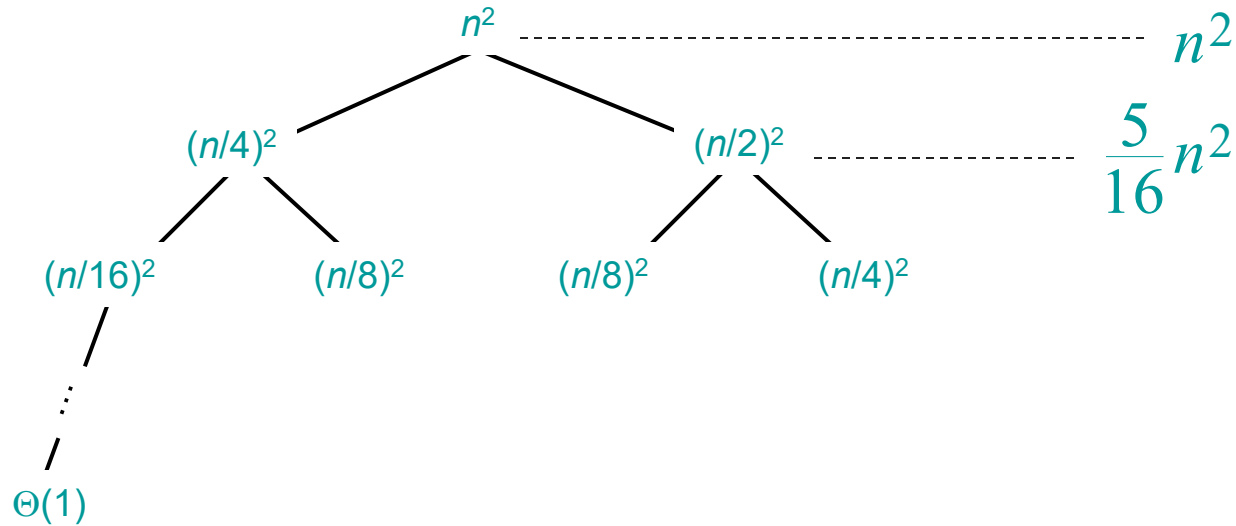
Tree Method

- Solve this recurrence relation using tree method: $T(n) = T\left(\frac{n}{4}\right) + T\left(\frac{n}{2}\right) + n^2$



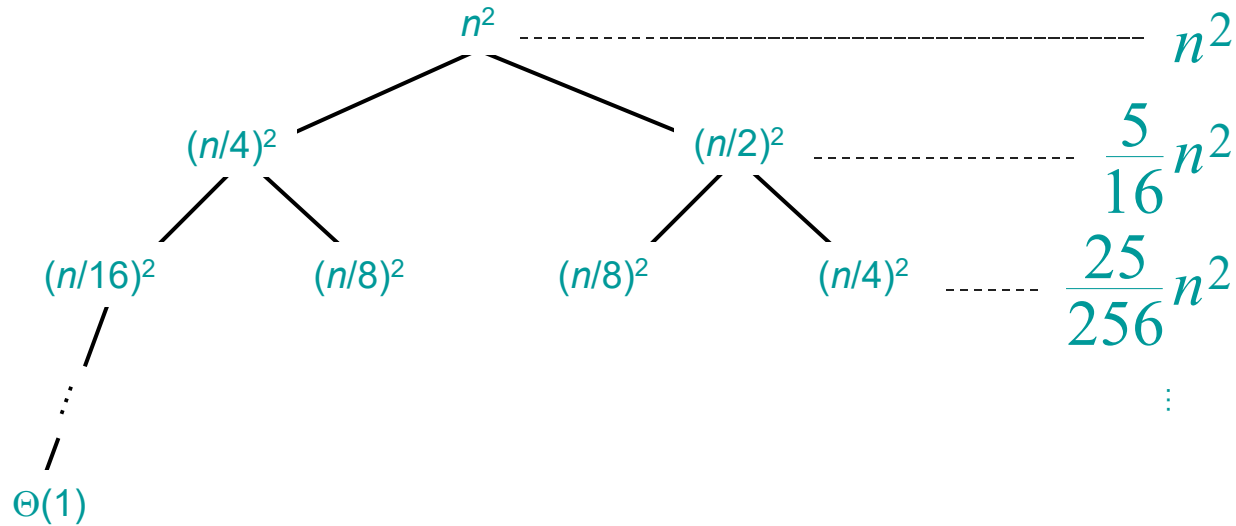
Tree Method

- Solve this recurrence relation using tree method: $T(n) = T\left(\frac{n}{4}\right) + T\left(\frac{n}{2}\right) + n^2$



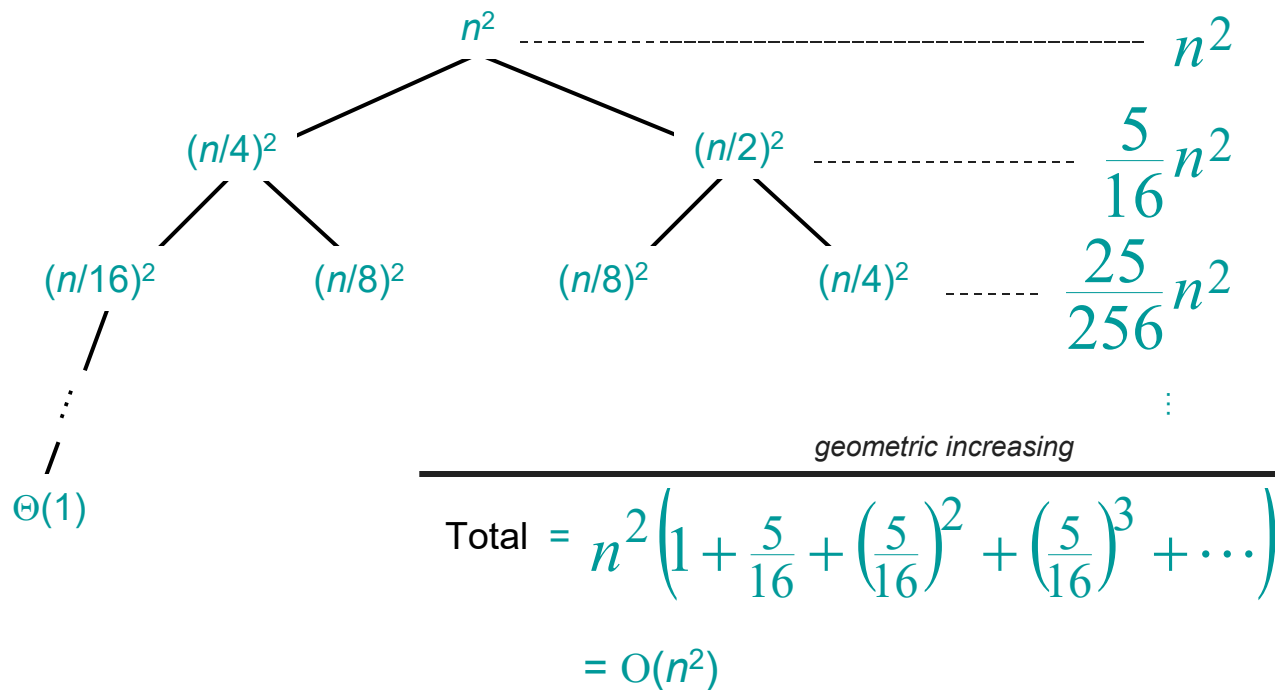
Tree Method

- Solve this recurrence relation using tree method: $T(n) = T\left(\frac{n}{4}\right) + T\left(\frac{n}{2}\right) + n^2$



Tree Method

- Solve this recurrence relation using tree method: $T(n) = T\left(\frac{n}{4}\right) + T\left(\frac{n}{2}\right) + n^2$



Tree Method

- Solve this recurrence relation using tree method: $T(n) = 2T\left(\frac{n}{2}\right) + 10$

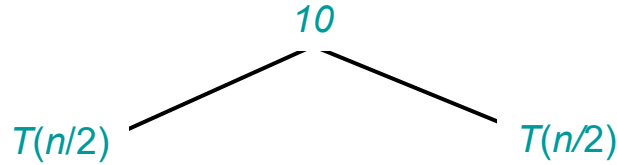
$T(n)$

Height of the tree=
 $\log_2 n$



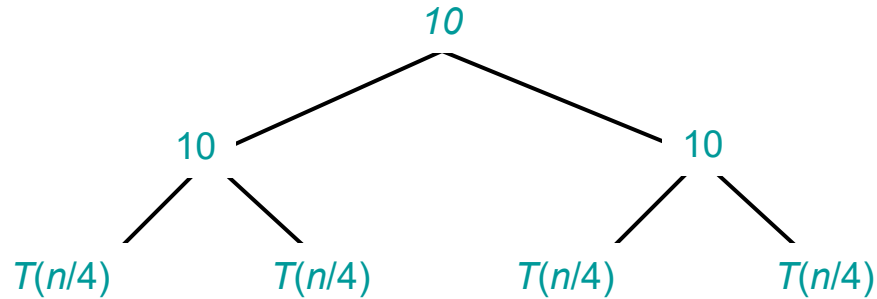
Tree Method

- Solve this recurrence relation using tree method: $T(n) = 2T\left(\frac{n}{2}\right) + 10$



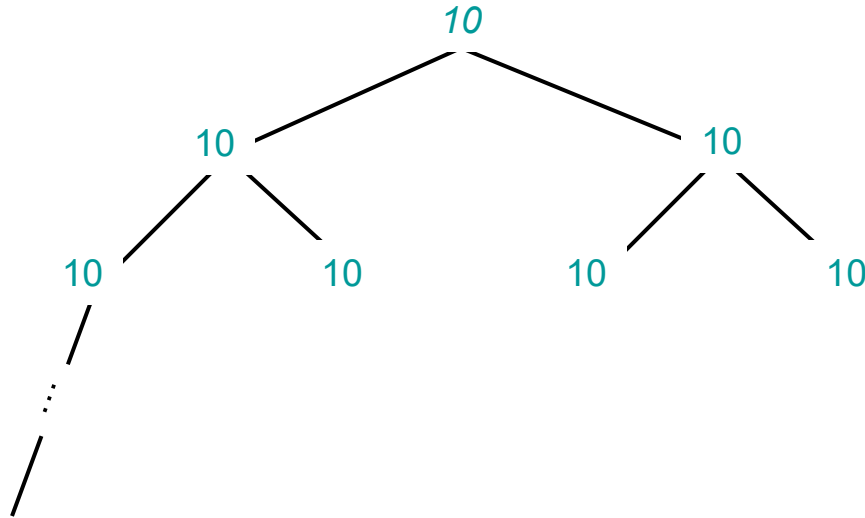
Tree Method

- Solve this recurrence relation using tree method: $T(n) = 2T\left(\frac{n}{2}\right) + 10$



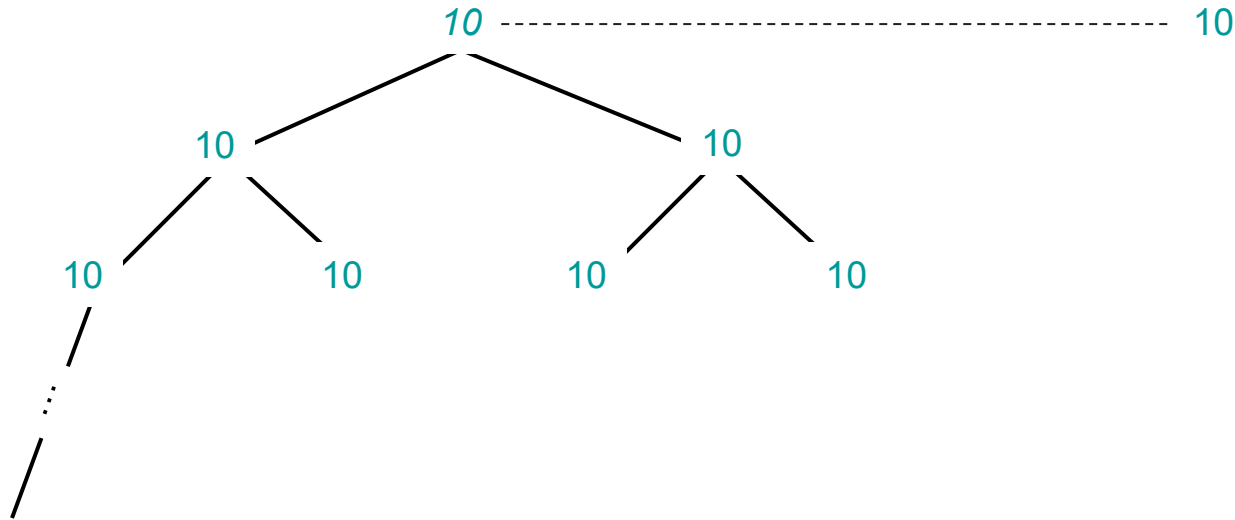
Tree Method

- Solve this recurrence relation using tree method: $T(n) = 2T\left(\frac{n}{2}\right) + 10$



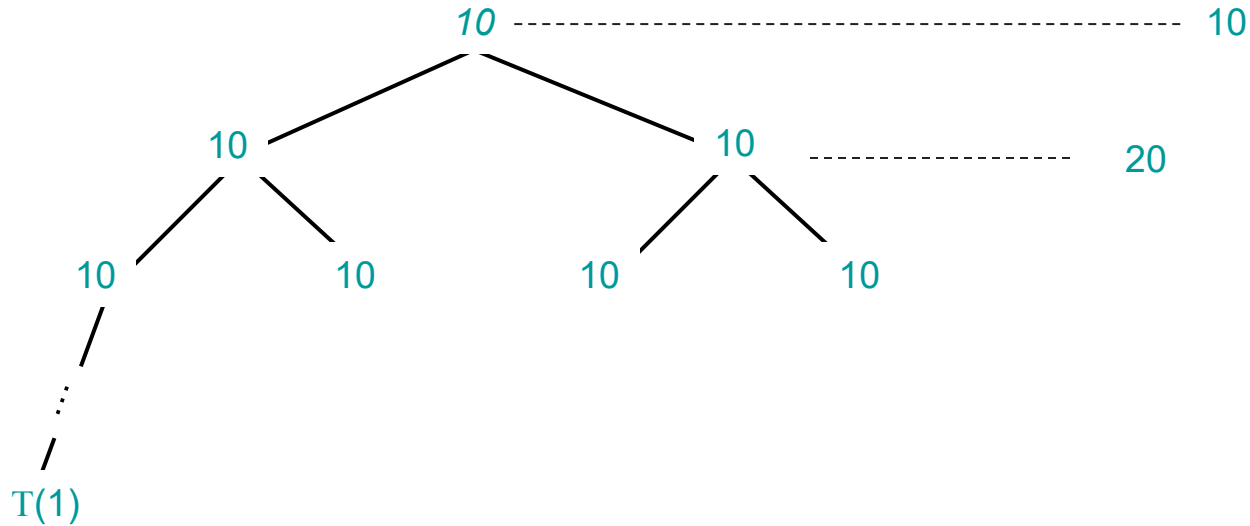
Tree Method

- Solve this recurrence relation using tree method: $T(n) = 2T\left(\frac{n}{2}\right) + 10$



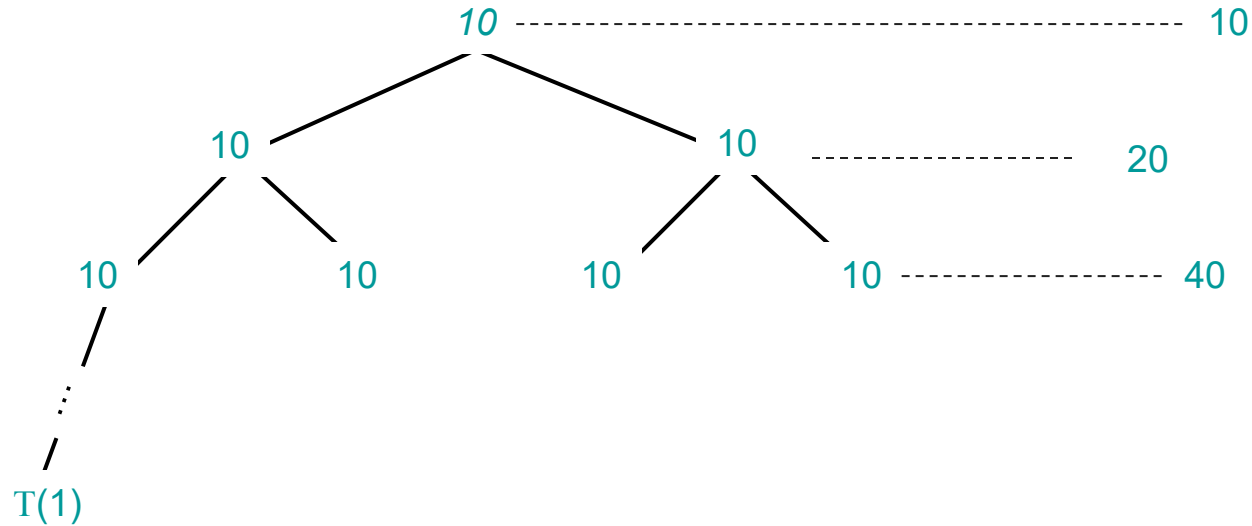
Tree Method

- Solve this recurrence relation using tree method: $T(n) = 2T\left(\frac{n}{2}\right) + 10$



Tree Method

- Solve this recurrence relation using tree method: $T(n) = 2T\left(\frac{n}{2}\right) + 10$



Tree Method

- Solve this recurrence relation using tree method: $T(n) = T(n) = 2T\left(\frac{n}{2}\right) + 10$

$$Total = \sum_{i=1}^{\log_2 n} 10 + 20 + 40 + 80 \quad \text{geometric increasing}$$

Where $a = 10, r = 2, h = \log_2 n$

$$\therefore Total = 10 \left(\frac{1 - 2^{\log_2 n}}{1 - 2} \right)$$

$$\therefore Total = 10 \left(\frac{1 - n}{1 - 2} \right) = O(n)$$



Tree Method

- Solve this recurrence relation using tree method: $T(n) = 3T\left(\frac{n}{3}\right) + n^2$

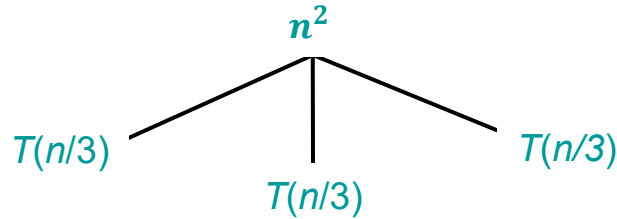
$T(n)$

Height of the tree=
 $\log_3 n$



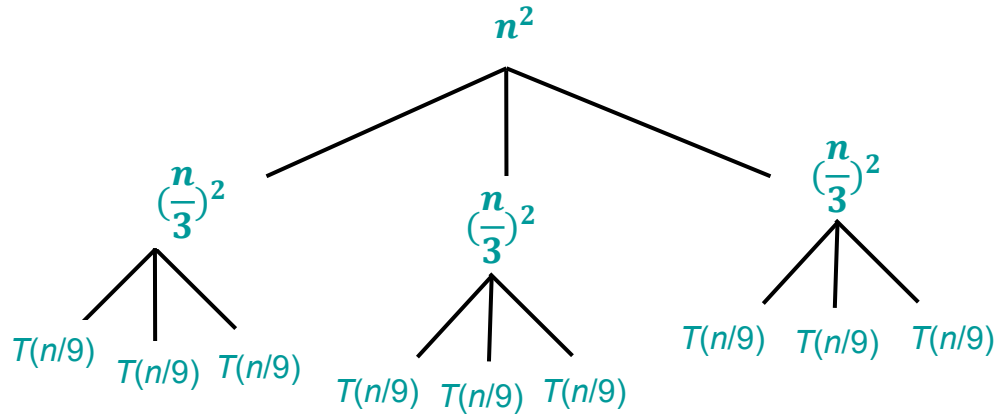
Tree Method

- Solve this recurrence relation using tree method: $T(n) = 3T\left(\frac{n}{3}\right) + n^2$



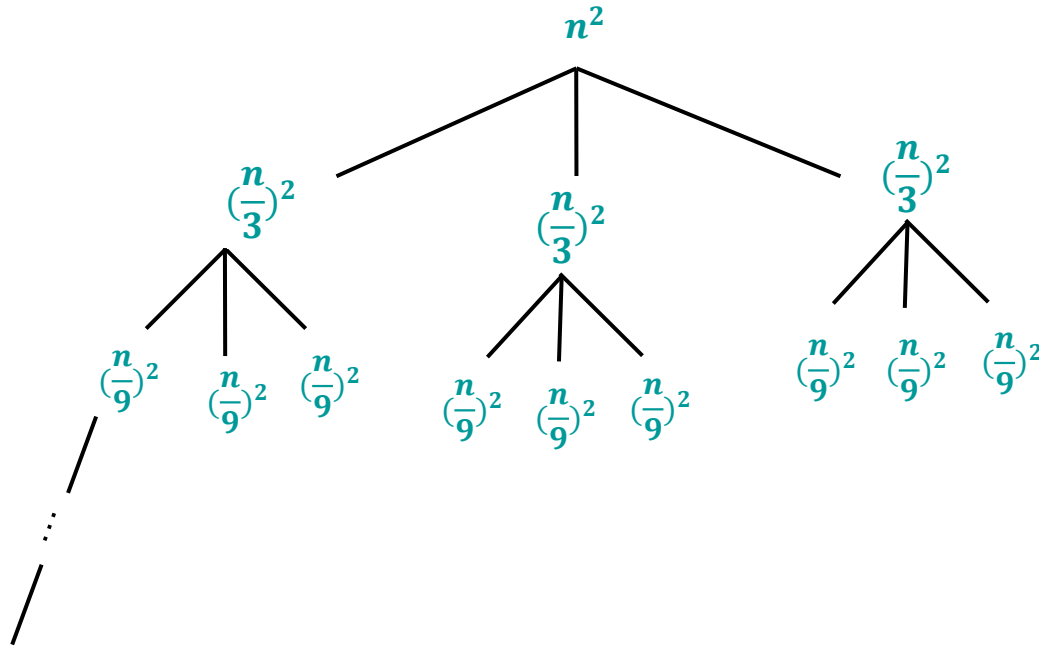
Tree Method

- Solve this recurrence relation using tree method: $T(n) = 3T\left(\frac{n}{3}\right) + n^2$



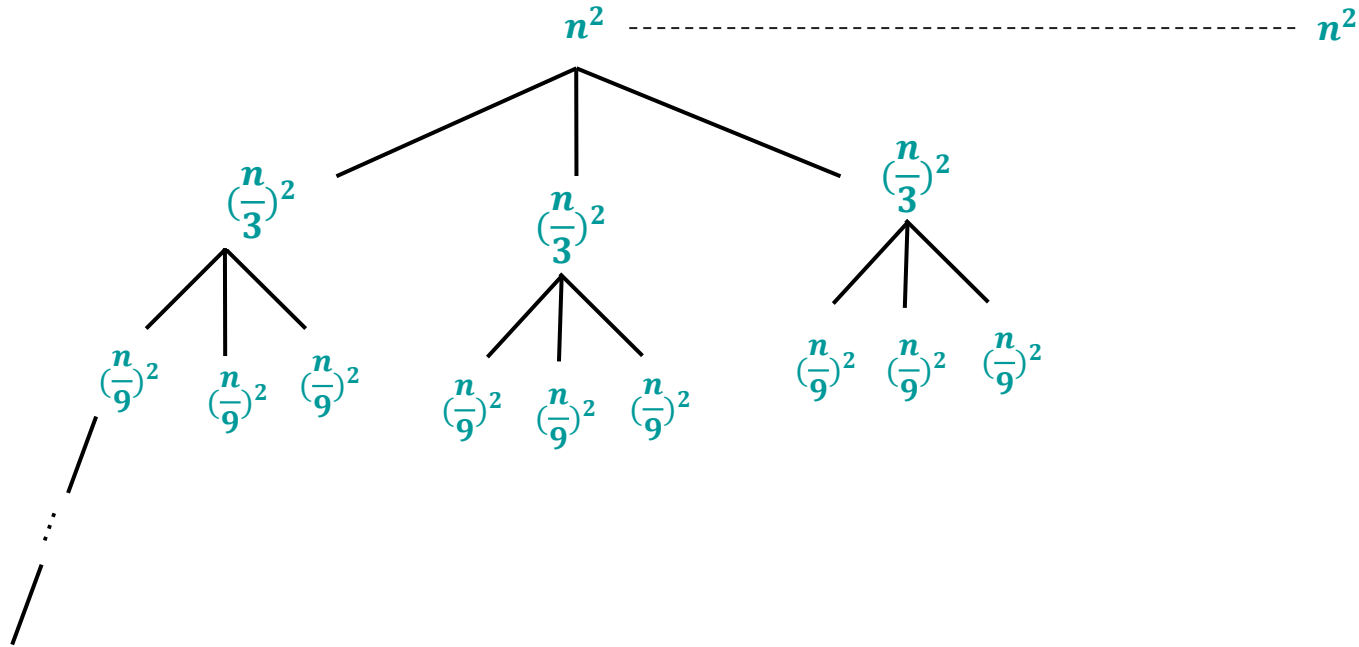
Tree Method

- Solve this recurrence relation using tree method: $T(n) = 3T\left(\frac{n}{3}\right) + n^2$



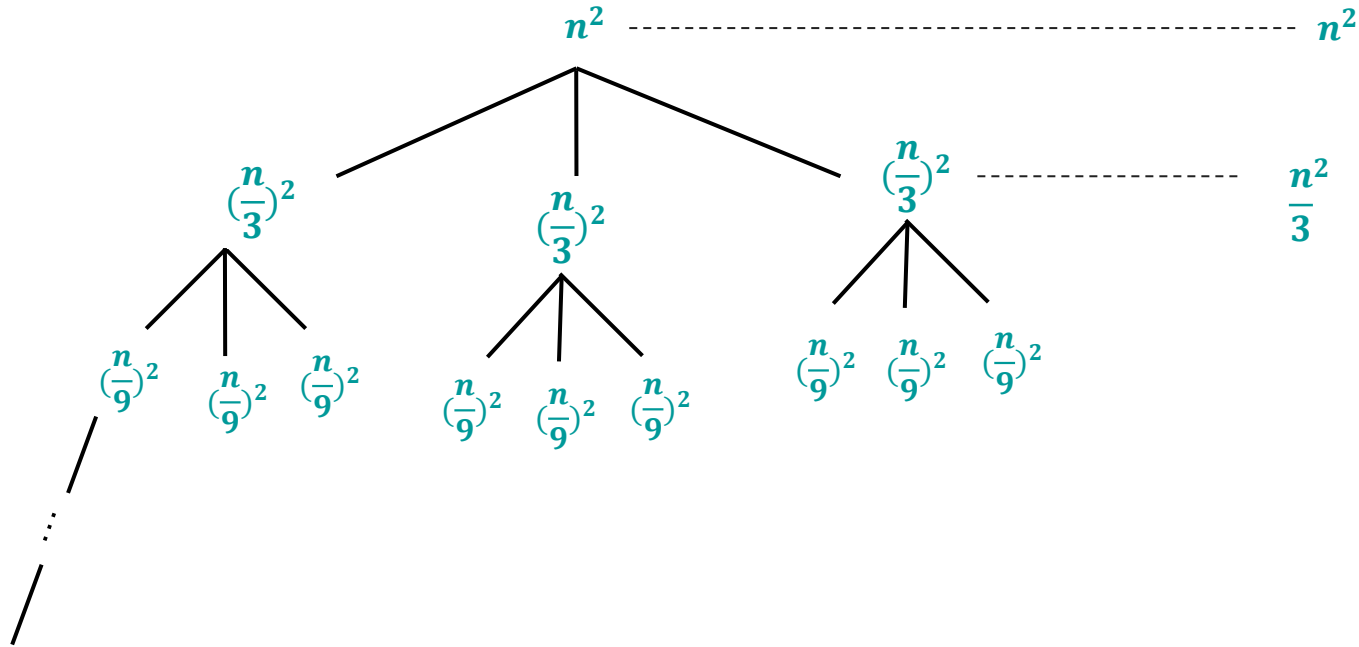
Tree Method

- Solve this recurrence relation using tree method: $T(n) = 3T\left(\frac{n}{3}\right) + n^2$



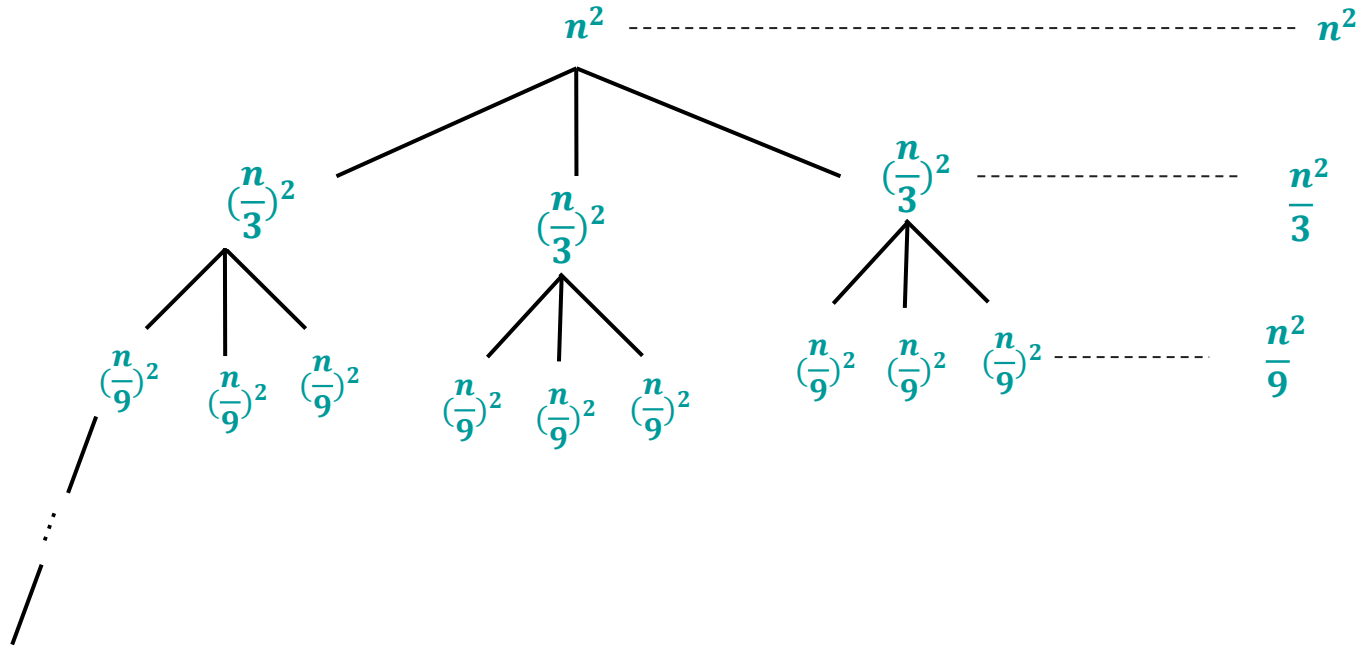
Tree Method

- Solve this recurrence relation using tree method: $T(n) = 3T\left(\frac{n}{3}\right) + n^2$



Tree Method

- Solve this recurrence relation using tree method: $T(n) = 3T\left(\frac{n}{3}\right) + n^2$



Tree Method

- Solve this recurrence relation using tree method: $T(n) = 3T\left(\frac{n}{3}\right) + n^2$

$$Total = \sum_{i=1}^{\log_3 n} n^2 + \frac{1}{3}n^2 + \frac{1}{9}n^2 + \frac{1}{27}n^2 \quad \text{geometric decreasing}$$

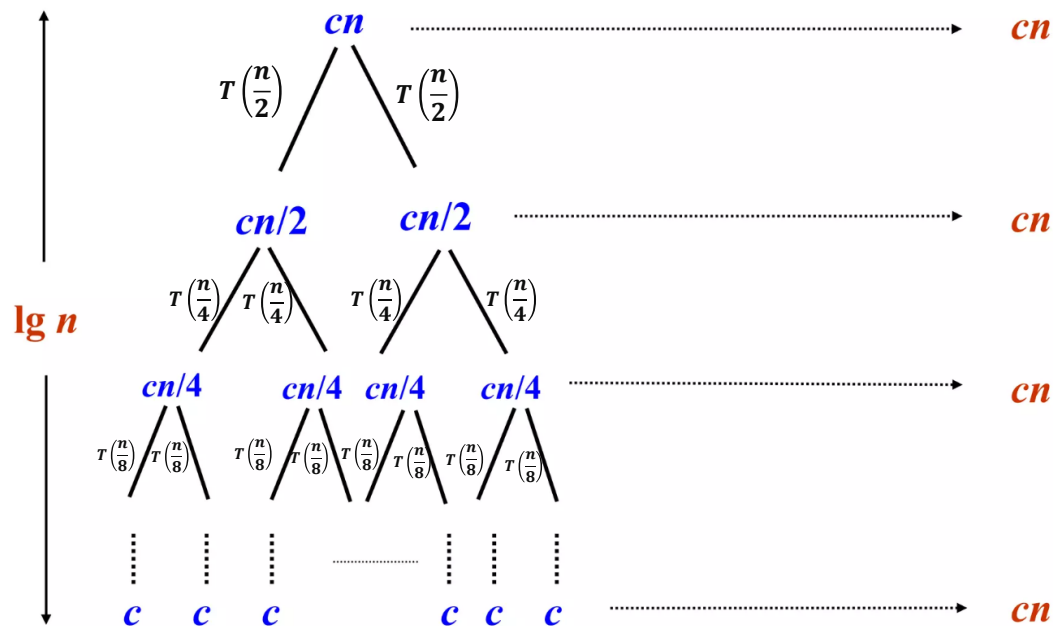
$$\text{Where } a = n^2, r = \frac{1}{3}, h = \log_3 n$$

$$\therefore Total = \frac{n^2}{1 - \frac{1}{3}} = O(n^2)$$



Tree Method

- Solve this recurrence relation using tree method: $T(n) = 2T\left(\frac{n}{2}\right) + cn$



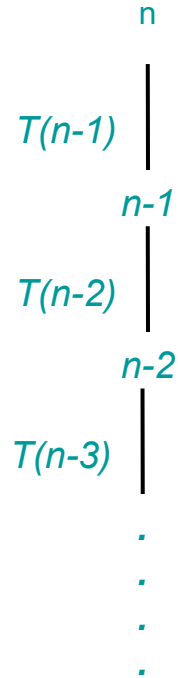
$$T(n) = \sum_{i=1}^{\log_2 n} cn = cn \log_2 n$$

$$T(n) = O(n \log_2 n)$$



Tree Method

- Solve this recurrence relation using tree method: $T(n) = T(n - 1) + n$



$$\text{Height} = n$$

$$T(n) = n + (n-1) + (n-2) + (n-3) + \dots + 3 + 2 + 1$$

$$\sum_{i=1}^n i = \frac{n(n+1)}{2} = O(n^2)$$



Let's solve this recurrence relation using substitution method: $T(n)=T(n-1)+n$



Substitution Method

- Solve this recurrence relation using substitution method: $T(n) = T(n - 1) + n$

$$\begin{aligned}T(n) &= T(n - 1) + n \\&= [T(n - 2) + (n - 1)] + n \\&= T(n - 2) + 2n - 1 \\&= [T(n - 3) + n - 2] + 2n - 1 = T(n - 3) + 3n - 3\end{aligned}$$

- After k substitution:

$$\begin{aligned}T(n) &= T(n - k) + kn - k \\ \text{Let assume } k &= n, \text{ and the base case } T(0) = 1 \\ T(n) &= T(0) + n * n - n \\ T(n) &= 1 + n^2 - n \\ \therefore \text{Time complexity: } &O(n^2)\end{aligned}$$



Let's solve this recurrence relation using master method: $T(n)=T(n-1)+n$



Master Method

- The **Master Theorem** applies to recurrences of the form:

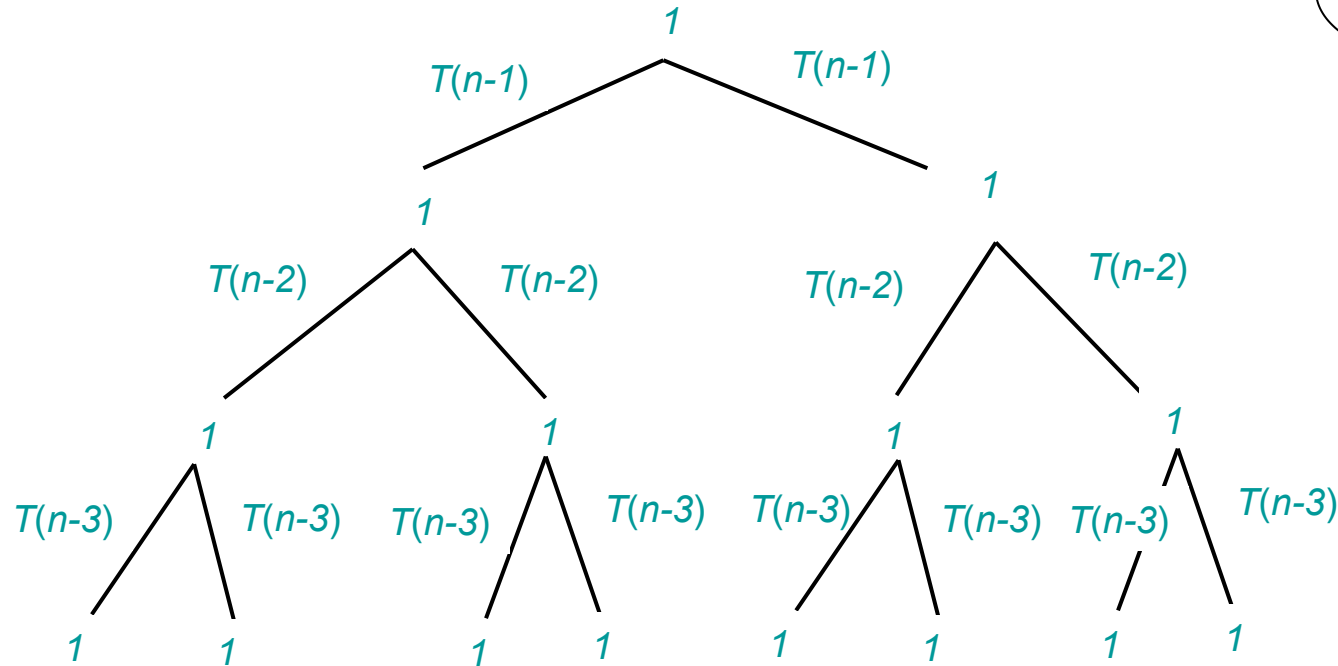
$$T(n) = aT\left(\frac{n}{b}\right) + f(n)$$

- This recurrence $T(n) = T(n - 1) + n$ is **not** of that divide-and-conquer form (it's a decreasing recurrence), so the standard Master Theorem **does not apply**.



Tree Method

- Solve this recurrence relation using tree method: $T(n) = 2T(n-1) + 1$



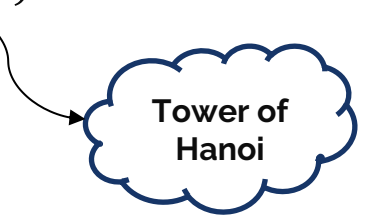
Tower of Hanoi



Tree Method

- Solve this recurrence relation using tree method: $T(n) = 2T(n-1) + 1$

$$Total = 1 + 2 + 4 + 8 \quad \text{geometric increasing}$$



Tower of Hanoi

$$\text{Where } a = 1, r = 2, h = n$$

$$\therefore Total = 1 \left(\frac{1 - 2^n}{1 - 2} \right) = O(2^n)$$



Let's solve this recurrence relation using substitution method: $T(n) = 2T(n - 1) + 1$



Substitution Method

- Solve this recurrence relation using substitution method: $T(n) = 2T(n - 1) + 1$
- This recurrence relation solved in the previous lecture pages 32-35, using geometric increasing.



Let's solve this recurrence relation using master method: $T(n) = 2T(n - 1) + 1$



Master Method

- The **Master Theorem** applies to recurrences of the form:

$$T(n) = aT\left(\frac{n}{b}\right) + f(n)$$

- This recurrence $T(n) = 2T(n - 1) + 1$ is **not** of that divide-and-conquer form (it's a decreasing recurrence), so the standard Master Theorem **does not apply**.



Any Question?

